

Hodnotící arch pro zkoušejícího

Tento dokument je určen pouze pro zkoušejícího. Nesdílejte ho s uchazečem.

Příprava prostředí

1. Spusťte službu: `npm start` (výchozí port 4010)
2. Ověřte dostupnost: `curl http://localhost:4010/health`
3. Připravte uchazeči notebook s nainstalovaným **Postmanem** a/nebo **SoapUI**
4. Předejte uchazeči soubor `interview-zadani.md` (nebo PDF verzi)
5. Volitelně importujte Postman kolekci z `postman/interview-mock.postman_collection.json`

API klíče pro uchazeče:

- Admin: `interview-key-2026`
- Read-only: `readonly-key-2026`

Pozor: Služba drží stav v paměti. Po restartu se data vrátí do výchozího stavu. Pokud uchazeč udělá chybu, může být potřeba restartovat službu.

Praktická část: Očekávané odpovědi

Úloha A: Zabezpečení API

A1 — Volání bez API klíče

```
GET /rest/interviews (bez X-API-Key) → 401
{ "error": "UNAUTHORIZED", "message": "Missing X-API-Key header." }
```

A2 — Neplatný API klíč

```
GET /rest/interviews (X-API-Key: invalid) → 401
{ "error": "UNAUTHORIZED", "message": "Invalid API key." }
```

A3 — Platný admin klíč

```
GET /rest/interviews (X-API-Key: interview-key-2026) → 200
{ "interviews": [...] }
```

A4 — Read-only klíč na čtení

```
GET /rest/interviews (X-API-Key: readonly-key-2026) → 200
{ "interviews": [...] }
```

A5 — Read-only klíč na zápis

```
POST /rest/interviews (X-API-Key: readonly-key-2026) → 403
{ "error": "FORBIDDEN", "message": "Read-only API key cannot perform write operations." }
```

A6 — Veřejné endpointy

```
GET /health (bez klíče) → 200
GET /soap?wsdl (bez klíče) → 200
```

A7 — SOAP bez autentizace

```
POST /soap (bez X-API-Key) → 401
{ "error": "UNAUTHORIZED", "message": "Missing X-API-Key header." }
```

Poznámka: Odpověď je JSON, ne XML — autentizace běží na úrovni middleware před SOAP parserem.

Úloha B: REST API

Všechny requesty s hlavičkou `X-API-Key: interview-key-2026`

B1 — Healthcheck

```
GET /health → 200
{
  "status": "ok",
  "service": "interview-mock",
  "timestamp": "2026-...",
  "protocols": ["REST", "SOAP"],
  "counts": { "interviews": 2, "candidates": 2 }
}
```

B2 — Seznam pohovorů

```
GET /rest/interviews → 200
Odpověď: { "interviews": [...] } – pole se 2 záznamy (INT-001, INT-002)
Každý záznam musí obsahovat vnořený objekt "candidate" (ne jen candidateId)
```

B3 — Filtrování

```
GET /rest/interviews?status=SCHEDULED → 200
Vrátí pouze INT-001 (status SCHEDULED)
```

B4 — Detail

```
GET /rest/interviews/INT-001 → 200
candidate.firstName = "Anna", candidate.lastName = "Novak"
candidate.skills = ["JavaScript", "REST", "SQL"]
```

B5 — Vytvoření

```
POST /rest/interviews
Content-Type: application/json
{ "candidateId": "CAND-002", "position": "Integration Tester", "scheduledAt": "2026-06-01T10:00:00.000Z" }
→ 201
Nové ID bude INT-003 (auto-increment)
```

B6 — Změna statusu

```
PATCH /rest/interviews/INT-003/status
Content-Type: application/json
{ "status": "IN_PROGRESS" }
→ 200
```

B7 — Hodnocení

```
POST /rest/interviews/INT-003/evaluate
Content-Type: application/json
```

```
{ "score": 68 }  
→ 200  
status = "COMPLETED", recommendation = "REVIEW" (68 je v rozmezí 55–74)
```

B8 — Negativní testy (uchazeč vybírá min. 2)

Scénář	Request	Očekávaná odpověď
Neexistující pohovor	GET /rest/interviews/INT-999	404, INTERVIEW_NOT_FOUND
Neexistující kandidát	POST /rest/interviews { "candidateId": "CAND-999" }	400, UNKNOWN_CANDIDATE
Nevalidní skóre (>100)	POST /rest/interviews/INT-001/evaluate { "score": 150 }	400, INVALID_SCORE
Nevalidní skóre (text)	POST /rest/interviews/INT-001/evaluate { "score": "abc" }	400, INVALID_SCORE
Neplatný status	PATCH /rest/interviews/INT-001/status { "status": "INVALID" }	400, INVALID_STATUS
Neexistující route	GET /rest/neexistuje	404, ROUTE_NOT_FOUND
Neexistující kandidát	GET /rest/candidates/CAND-999	404, CANDIDATE_NOT_FOUND
Interní chyba serveru	(nelze snadno vyvolat)	500, INTERNAL_ERROR

Úloha C: Řetězení parametrů a práce s proměnnými

Tato úloha ověřuje, zda uchazeč umí **dynamicky pracovat s daty z odpovědí** a správně používat **scoping proměnných** — klíčové dovednosti integračního testera.

C1 — Nastavení proměnných prostředí

Uchazeč by měl vytvořit environment a nastavit `baseUrl` + `apiKey`. Hodnotí se zdůvodnění:

Proměnná	Správná úroveň	Zdůvodnění
<code>baseUrl</code>	Environment	Mění se mezi prostředími (test/staging/prod)
<code>apiKey</code>	Environment	Jiný klíč pro různá prostředí, nepatří do globálních (bezpečnost)
<code>candidateId</code>	Collection	Dočasná hodnota získaná z odpovědi, platná jen v rámci test flow
<code>interviewId</code>	Collection	Dočasná hodnota získaná z odpovědi, platná jen v rámci test flow

Správné odpovědi na dotaz „proč ne globální?“:

- Globální proměnné se sdílejí napříč kolekcemi — hrozí kolize názvů a nechtěné přepisování
- API klíče v globálních proměnných jsou bezpečnostní riziko (viditelné všude)
- Environment proměnné umožňují přepínat mezi prostředími jedním kliknutím

V SoapUI odpovídající úrovně:

- Global Properties = globální proměnné Postmanu
- Project Properties = environment proměnné Postmanu
- Test Suite / Test Case Properties = collection / lokální proměnné Postmanu

C2 — Vyhledání kandidáta

```
GET /rest/candidates?email=petr.svoboda@example.test → 200  
{ "candidates": [{ "id": "CAND-002", "firstName": "Petr", ... }] }
```

Uchazeč si musí uložit ID do **collection proměnné** (ne globální!).

V Postmanu (Tests tab):

```
const data = pm.response.json();
pm.collectionVariables.set("candidateId", data.candidates[0].id);
```

V SoapUI: Property Transfer step nebo Groovy skript do Test Case Properties.

C3 — Vytvoření pohovoru s dynamickým ID

```
POST /rest/interviews
{ "candidateId": "{{candidateId}}", "position": "Senior Integration Tester", "scheduledAt": "2026-07-01T09:00:00.000Z" }
→ 201
```

Klíčové: uchazeč nesmí hardcodovat CAND-002, ale použít proměnnou z C2.
Nové ID bude INT-004 (pokud INT-003 vzniklo v úloze B).

Uchazeč uloží `interviewId` z odpovědi do collection proměnné:

```
const data = pm.response.json();
pm.collectionVariables.set("interviewId", data.interview.id);
```

C4 — Ověření vytvoření

```
GET /rest/interviews/{{interviewId}} → 200
```

Uchazeč musí použít proměnnou z C3.
Ověřit: `candidate.firstName = "Petr", position = "Senior Integration Tester"`

C5 — Lifecycle

```
PATCH /rest/interviews/{{interviewId}}/status { "status": "IN_PROGRESS" } → 200
POST /rest/interviews/{{interviewId}}/evaluate { "score": 91 } → 200
recommendation = "HIRE" (91 >= 75)
```

C6 — Filtrování s dynamickým parametrem

```
GET /rest/interviews?candidateId={{candidateId}}&status=COMPLETED → 200
```

Musí vrátit pohovor vytvořený v C3 (po hodnocení v C5).

C7 — Vyhledání podle skillu

```
GET /rest/candidates?skill=SQL → 200
```

Vrátí 1 kandidáta: CAND-001 (Anna Novak, skills: JavaScript, REST, SQL)

Na co se zaměřit:

Co sledovat	Vynikající	Dostatečné	Nedostatečné
Proměnné	Automatické ukládání z odpovědí (skripty)	Ruční kopírování hodnot	Hardcoded hodnoty
Scoping	Správně rozlišuje global/env/collection	Vše v jedné úrovni, ale funguje	Nepoužívá proměnné
Zdůvodnění	Vysvětlí bezpečnost, přenositelnost, kolize	Částečně vysvětlí	Neumí vysvětlit

Úloha D: SOAP API

Všechny SOAP requesty s hlavičkou `X-API-Key: interview-key-2026`

D1 — WSDL

GET /soap?wsdl → 200, Content-Type: application/xml (bez autentizace)
Uchazeč by měl identifikovat 4 operace: ListInterviews, GetInterview, CreateInterview, UpdateInterviewStatus

D2 — ListInterviews

```
POST /soap (Content-Type: text/xml, X-API-Key: interview-key-2026)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:ListInterviewsRequest/>
  </soap:Body>
</soap:Envelope>
```

Odpověď: XML s ListInterviewsResponse , obsahuje všechny pohovory (včetně těch vytvořených v úlohách B a C).

D3 — GetInterview

```
POST /soap (Content-Type: text/xml, X-API-Key: interview-key-2026)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:GetInterviewRequest>
      <tns:id>INT-002</tns:id>
    </tns:GetInterviewRequest>
  </soap:Body>
</soap:Envelope>
```

Odpověď: score=82, recommendation=HIRE, candidate.firstName=Petr

D4 — CreateInterview

```
POST /soap (Content-Type: text/xml, X-API-Key: interview-key-2026)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:CreateInterviewRequest>
      <tns:candidateId>CAND-001</tns:candidateId>
      <tns:position>SOAP Tester</tns:position>
      <tns:scheduledAt>2026-06-15T14:00:00.000Z</tns:scheduledAt>
    </tns:CreateInterviewRequest>
  </soap:Body>
</soap:Envelope>
```

Odpověď: nové ID (INT-005 pokud INT-003 vzniklo v B a INT-004 v C).

D5 — UpdateInterviewStatus

```
POST /soap (Content-Type: text/xml, X-API-Key: interview-key-2026)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-mock">
  <soap:Body>
    <tns:UpdateInterviewStatusRequest>
```

```
<tns:id>INT-005</tns:id>
<tns:status>CANCELLED</tns:status>
</tns:UpdateInterviewStatusRequest>
</soap:Body>
</soap:Envelope>
```

Úloha E: Cross-protocol ověření

Uchazeč by měl zavolat `GET /rest/interviews` a najít v odpovědi všechny nově vytvořené pohovory (z úloh B, C i D). To dokazuje, že obě API sdílejí stejný datový stav.

Bonusový bod: uchazeč zmíní návrhy na vylepšení zabezpečení (rate limiting, HTTPS, JWT místo API klíčů, audit log apod.).

Teoretická část: Očekávané odpovědi

REST otázky

1. REST principy

- Representational State Transfer
- Klíčové principy: client-server, stateless, cacheable, uniform interface, layered system
- Zdroje identifikované URL, manipulace přes HTTP metody

2. PUT vs PATCH

- PUT nahrazuje celý zdroj (úplná reprezentace)
- PATCH aplikuje částečnou modifikaci (pouze změněná pole)
- PUT je idempotentní, PATCH nemusí být (ale v praxi často je)

3. Idempotence

- Opakované volání se stejnými parametry má stejný výsledek jako jedno volání
- Idempotentní: GET, PUT, DELETE, HEAD, OPTIONS
- Ne-idempotentní: POST (vytváří nový zdroj při každém volání)

4. HTTP status kódy

- 1xx: Informační (100 Continue, 101 Switching Protocols)
- 2xx: Úspěch (200 OK, 201 Created, 204 No Content)
- 3xx: Přesměrování (301 Moved Permanently, 302 Found, 304 Not Modified)
- 4xx: Chyba klienta (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found)
- 5xx: Chyba serveru (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable)

5. OpenAPI/Swagger

- Standardizovaný formát popisu REST API (YAML/JSON)
- Definuje endpointy, metody, parametry, request/response schémata
- Umožňuje generování dokumentace, klientů, serverových stubů a testů

6. Autentizace/autorizace

- API klíče: jednoduché, v headeru nebo query parametru
- OAuth 2.0: autorizační framework s tokeny (access/refresh)
- JWT: self-contained tokeny s claims, podepsané
- Testování: platný/neplatný/expirovaný token, nedostatečná oprávnění

7. Content Negotiation

- Mechanismus HTTP pro dohodnutí formátu odpovědi
- Klient posílá `Accept` header (např. `application/json`, `application/xml`)
- Server posílá `Content-Type` header s formátem odpovědi

SOAP otázky

8. SOAP struktura

- Simple Object Access Protocol
- Envelope: kořenový element, definuje namespaces
- Header: volitelný, metadata (autentizace, routing)
- Body: povinný, obsahuje samotnou zprávu/operaci
- Fault: volitelný (v Body), chybové informace

9. WSDL

- Web Services Description Language
- Obsahuje: typy (XSD schémata), zprávy, operace, port types, bindings, services
- Definuje „kontrakt“ služby — jaké operace jsou dostupné a jak je volat

10. Document/literal vs RPC/encoded

- Document/literal: XML dokument v Body, validovatelný přes XSD, moderní standard
- RPC/encoded: vzdálené volání procedury, parametry jako elementy s typy
- Document/literal je dnes preferovaný (WS-I Basic Profile)

11. SOAP Fault

- Element v SOAP Body pro chybové odpovědi
- Struktura: faultcode (kód chyby), faultstring (popis), detail (dodatečné info)
- SOAP 1.2: Code, Reason, Detail, Node, Role

12. SOAP vs REST

- SOAP výhody: formální kontrakt (WSDL), WS-Security, transakce (WS-AtomicTransaction), spolehlivost (WS-ReliableMessaging)
- SOAP nevýhody: verbose XML, složitější implementace, horší výkon
- REST výhody: jednoduchost, flexibilní formáty (JSON), cachování, široká podpora
- REST nevýhody: žádný formální kontrakt (OpenAPI je volitelný), méně standardizovaná bezpečnost

Zabezpečení a obecné otázky

13. Integrační testování

- Testuje interakci mezi komponentami/systémy
- Unit: izolovaná jednotka kódu
- Integrační: spolupráce více komponent, reálné závislosti
- E2E: celý systém end-to-end včetně UI

14. Autentizace vs autorizace

- Autentizace: ověření identity (kdo jsi?) — login, API klíč, certifikát
- Autorizace: ověření oprávnění (co smíš?) — role, permissions, policies
- Příklad z praxe: autentizace = přihlášení API klíčem; autorizace = read-only klíč nemůže vytvářet záznamy (403)

15. Contract testing

- Ověřuje kompatibilitu API mezi consumer a provider
- Pact: consumer-driven contracts
- Vhodné pro microservices, kdy více týmů vyvíjí nezávisle
- Consumer definuje očekávání, provider je verifikuje

16. Výkonnostní testování

- Zátěžové testy: chování pod zátěží (JMeter, k6, Gatling)
- Stress testy: hledání bodu zlomu
- Soak testy: dlouhodobá stabilita

- Metriky: response time, throughput, error rate, percentily (p50, p95, p99)

17. Pozitivní vs negativní testování

- Pozitivní: validní vstupy → očekávaný úspěšný výsledek
- Negativní: nevalidní vstupy → správná chybová odpověď
- API příklady: pozitivní = validní JSON → 201; negativní = chybějící pole → 400

18. Mocky a stuby

- Stub: vrací předem definované odpovědi (pasivní)
- Mock: ověřuje, že byl zavolán se správnými parametry (aktivní, assertions)
- Vhodné: izolace od externích závislostí, nestabilní služby, vývoj paralelně

19. Způsoby zabezpečení API

- API klíče: jednoduché, vhodné pro server-to-server, nelze omezit scope
- OAuth 2.0: standard pro delegovanou autorizaci, access/refresh tokeny, scopes
- JWT: JSON Web Token, self-contained (nemusí se volat auth server), podepsané (HMAC/RSA)
- Porovnání: API klíče = nejjednodušší ale nejméně bezpečné; OAuth = nejrobustnější ale nejsložitější; JWT = dobrý kompromis

20. CORS

- Cross-Origin Resource Sharing
- Mechanismus prohlížeče pro povolení/zakázání cross-origin HTTP requestů
- Preflight request (OPTIONS) pro kontrolu povolených origin, metod, hlaviček
- Důležitý pro webové API volané z frontendu na jiné doméně
- Access-Control-Allow-Origin, Access-Control-Allow-Methods, Access-Control-Allow-Headers

21. Scoping proměnných v Postmanu / SoapUI

Úroveň (Postman)	Úroveň (SoapUI)	Viditelnost	Příklad použití
Global	Global Properties	Všechny kolekce, všechna prostředí	Společné konstanty (timeout, verze API) — používat minimálně
Environment	Project Properties	Všechny kolekce v daném prostředí	baseUrl, apiKey — mění se mezi test/staging/prod
Collection	Test Suite Properties	Jen v rámci kolekce	Sdílená data mezi requesty (token získaný při loginu)
Local (data/pre-req)	Test Case Properties	Jen jeden request / test step	Dočasné hodnoty: candidateId z odpovědi, interviewId

- Pořadí přepisování (Postman): Local > Data > Environment > Collection > Global
- Bezpečnostní pravidlo: citlivé hodnoty (klíče, tokeny) patří do Environment, nikdy do Global
- SoapUI navíc má Test Step Properties (nejnižší úroveň) a Property Transfer step pro automatické předávání hodnot mezi kroky

Doporučený průběh pohovoru

1. **Úvod (5 min)** — představení, vysvětlení formátu, předání API klíčů
2. **Praktická část (75 min)** — uchazeč pracuje, zkoušející pozoruje a odpovídá na dotazy k zadání
3. **Teoretická část (15 min)** — zkoušející vybere 2 REST + 2 SOAP + 2 obecné/security otázky
4. **Diskuze (10 min)** — uchazeč prezentuje shrnutí, zkoušející se ptá na postřehy
5. **Závěr (5 min)** — prostor pro otázky uchazeče

Na co se zaměřit při pozorování:

- Systematičnost přístupu (nejdřív si přečte dokumentaci, nebo rovnou zkouší?)
- Kvalita assertions (ověřuje jen status kód, nebo i tělo odpovědi?)
- Organizace práce (pojmenovávání requestů, struktura kolekce)
- Schopnost interpretovat chybové odpovědi
- Práce s WSDL a XML (zvládá namespace, strukturu envelope)
- **Práce s autentizací** (jak rychle pochopí rozdíl admin vs reader klíč?)
- **Řetězení parametrů** (používá proměnné/property transfer, nebo kopíruje ručně?)
- **Scoping proměnných** (rozlišuje global/env/collection, nebo vše hází do jedné úrovně?)
- Komunikace — jak popisuje nalezené problémy